



**Politecnico
di Torino**
International
University

Progetto OS161 B: Implementazione Virtual Memory

Gruppo 42

- Greco Eleonora (s354761)
- Jafrancesco Chiara (s355023)
- Lemerle Stefano (s353334)

Esecuzione Utente & Gestione Processi

Esecuzione Programmi Utente (runprogram.c):

- Supporto al passaggio di argomenti (argc, argv) da linea di comando.
- Copia sicura tramite copyoutstr e allineamento dello stack pointer a 8 byte.

Estensione della struct proc & Process Table:

- Aggiunti campi chiave per la sincronizzazione: PID, Exit Code, semaforo e puntatore al padre.
- Introduzione della Process Table globale protetta da spinlock.

Esecuzione Utente & Gestione Processi

System Call Implementate:

- `sys_getpid`: ritorna l'identificatore del processo attuale.
- `sys_fork`: duplica il processo corrente creando un processo figlio identico con spazio di indirizzamento separato e restituendo il PID del figlio al padre.
- `sys_exit`: termina l'esecuzione di un processo.
- `sys_waitpid`: Sospende il processo chiamante finchè il processo figlio di cui è stato specificato il PID non ritorna (uso di `proc_wait`).
- `sys_sbrk`: modifica la dimensione dell'area heap del processo utente chiamante (demand paging)

L'Architettura Fisico-Logica: Coremap e Swap

La Coremap (Gestione RAM):

- Array di struct `coremap_entry` indicizzato per frame.
- Stati tracciati: `FREE`, `IN_USE`, `FIXED` (kernel e I/O bloccante).
- Politica di rimpiazzamento: FIFO basata su contatore incrementale.
- Canale di attesa `eviction_wchan` per prevenire race condition durante la deallocazione di pagine in I/O.

Interfaccia di Swap (Disco `lhd1`):

- Gestione tramite bitmap e mutua esclusione (`swap_lock`).

L'Architettura Fisico-Logica: Coremap e Swap

Funzioni di basso livello dedicate

- `swap_bootstrap`: Connette il disco chiamando `vfs_swapon`, ne ottiene la dimensione e inizializza la bitmap.
- `swap_alloc`: trova e riserva uno slot libero nella bitmap impostandolo a 1 sotto la protezione di `swap_lock`.
- `swap_free`: rilascia lo slot impostandolo a 0.
- `swap_write`: effettua la scrittura di una pagina dalla RAM al disco.
- `swap_read`: effettua la lettura di una pagina dalla disco al RAM.

L'Architettura Fisico-Logica: Coremap e Swap

Funzioni di basso livello dedicate

- `swap_bootstrap`: Connette il disco chiamando `vfs_swapon`, ne ottiene la dimensione e inizializza la bitmap.
- `swap_alloc`: trova e riserva uno slot libero nella bitmap impostandolo a 1 sotto la protezione di `swap_lock`.
- `swap_free`: rilascia lo slot impostandolo a 0.
- `swap_write`: effettua la scrittura di una pagina dalla RAM al disco.
- `swap_read`: effettua la lettura di una pagina dalla disco al RAM.

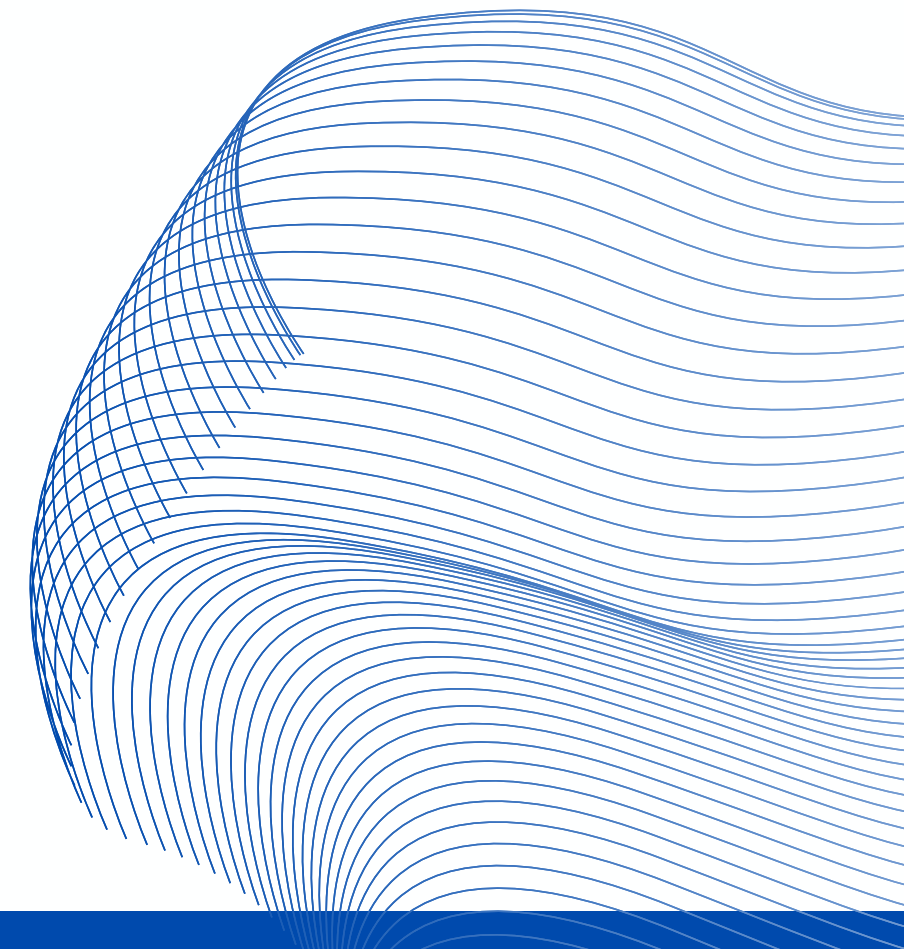
Il fulcro della VM: Page Table

Struttura Dinamica:

- Livello 1 (Page Directory): 1024 puntatori a tabelle di secondo livello (allocate on-demand).
- Livello 2 (Page Table): 1024 entry (paddr_t).

Stati di una Page Table Entry (PTE):

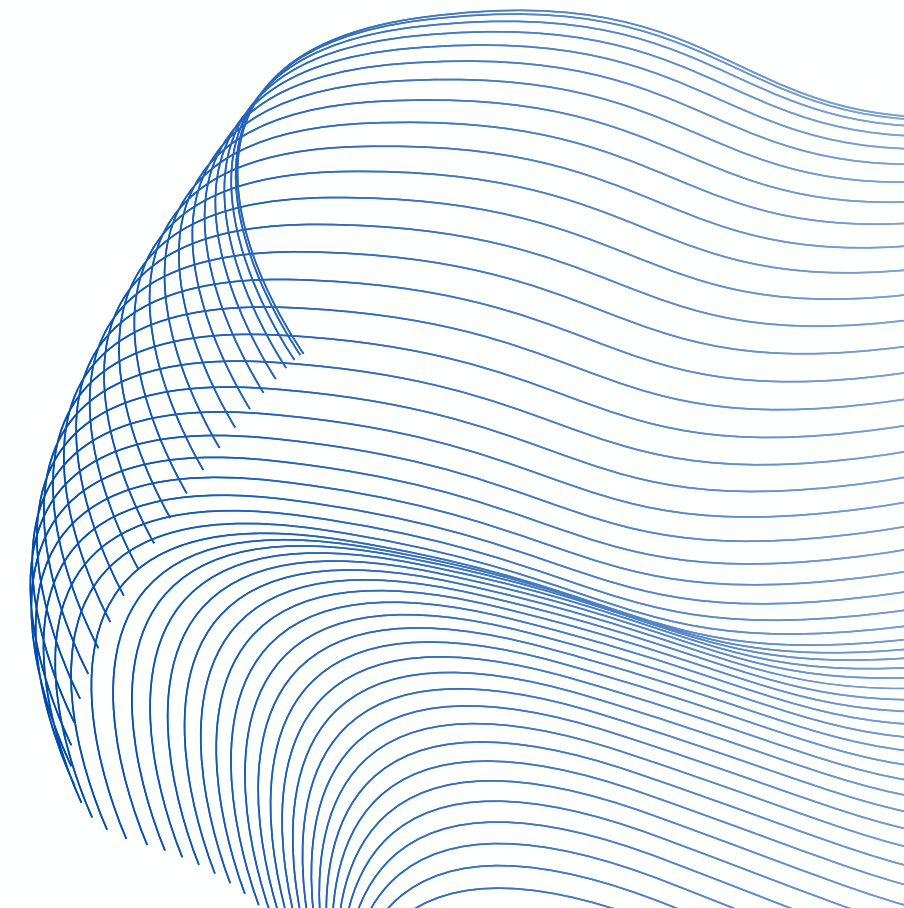
- 0x0: pagina non ancora allocata.
- PTE_PRESENT (0x1): pagina residente in RAM.
- PTE_SWAPPED (0x2) pagina salvata su disco (i bit alti contengono lo slot di swap).



Fulcro della VM:Page Table

Funzioni chiave

- `pagetable_create/pagetable_create_lv2`: crea una page directory di primo/secondo livello.
- `pagetable_destroy`: si occupa di dellocare la page table. Offre sincronizzazione tramite `mem_lock` e distruzione sicura attendendo il risveglio da `eviction_wchan`.
- `pagetable_copy`: deep copy della pagetable dell'address space padre all'address space figlio.
- `pagetable_get_entry`: restituisce un puntatore a `paddr_t`.



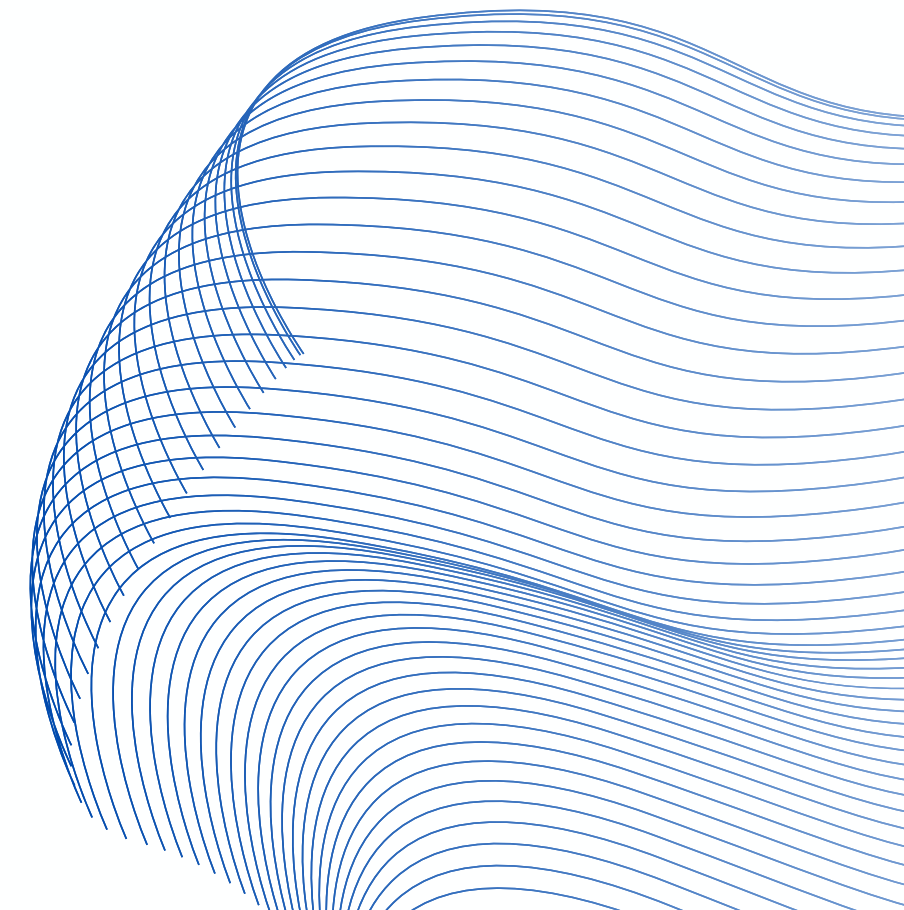
Traduzione:vm_fault e pagetable_translate

vm_fault (Gestore Centrale):

- Valida l'indirizzo e i permessi (readable, writeable, executable) confrontandoli con la lista concatenata delle regioni dell'address space.
- Aggiorna il TLB (gestendo slot vuoti o rimpiazzamento tlb_random).

pagetable_translate:

- Caso 1 (In RAM): Ritorna l'indirizzo fisico.
- Caso 2 (entry = 0): alloca un frame (azzerato con bzero) e chiama coremap_set_owner.
- Caso 3 (In Swap): alloca un frame, esegue swap_read dal disco, libera lo slot sulla bitmap dello swap e aggiorna la PTE a PTE_PRESENT.



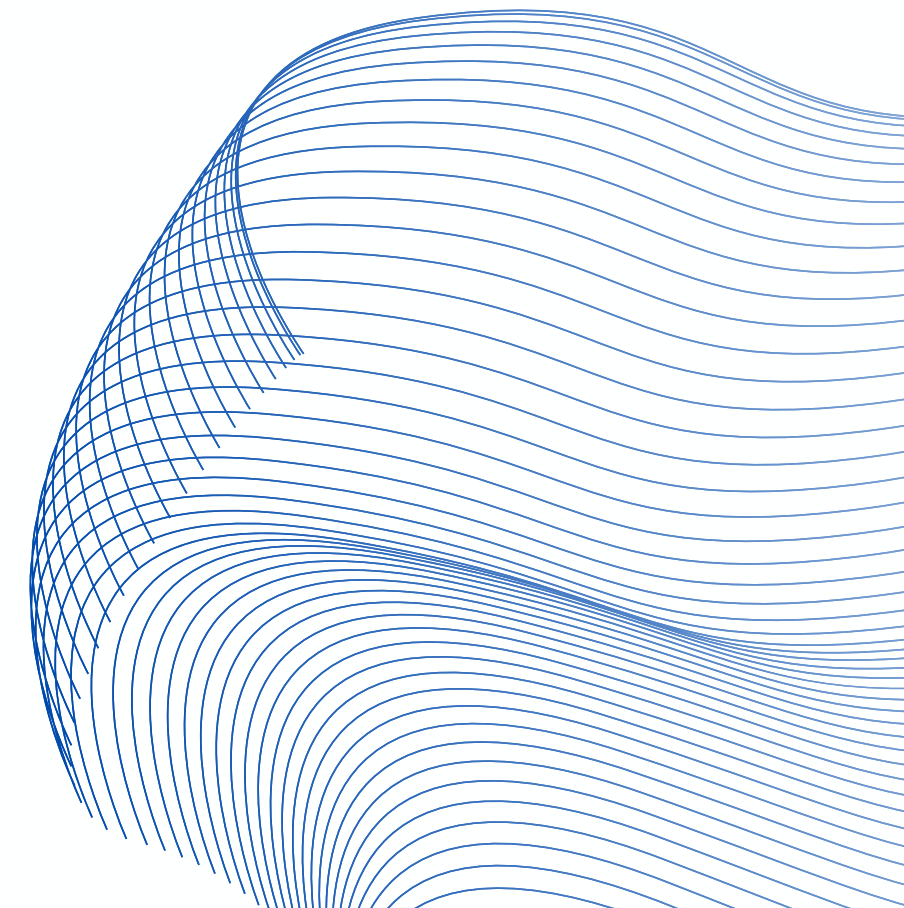
Traduzione:vm_fault e pagetable_translate

vm_fault (Gestore Centrale):

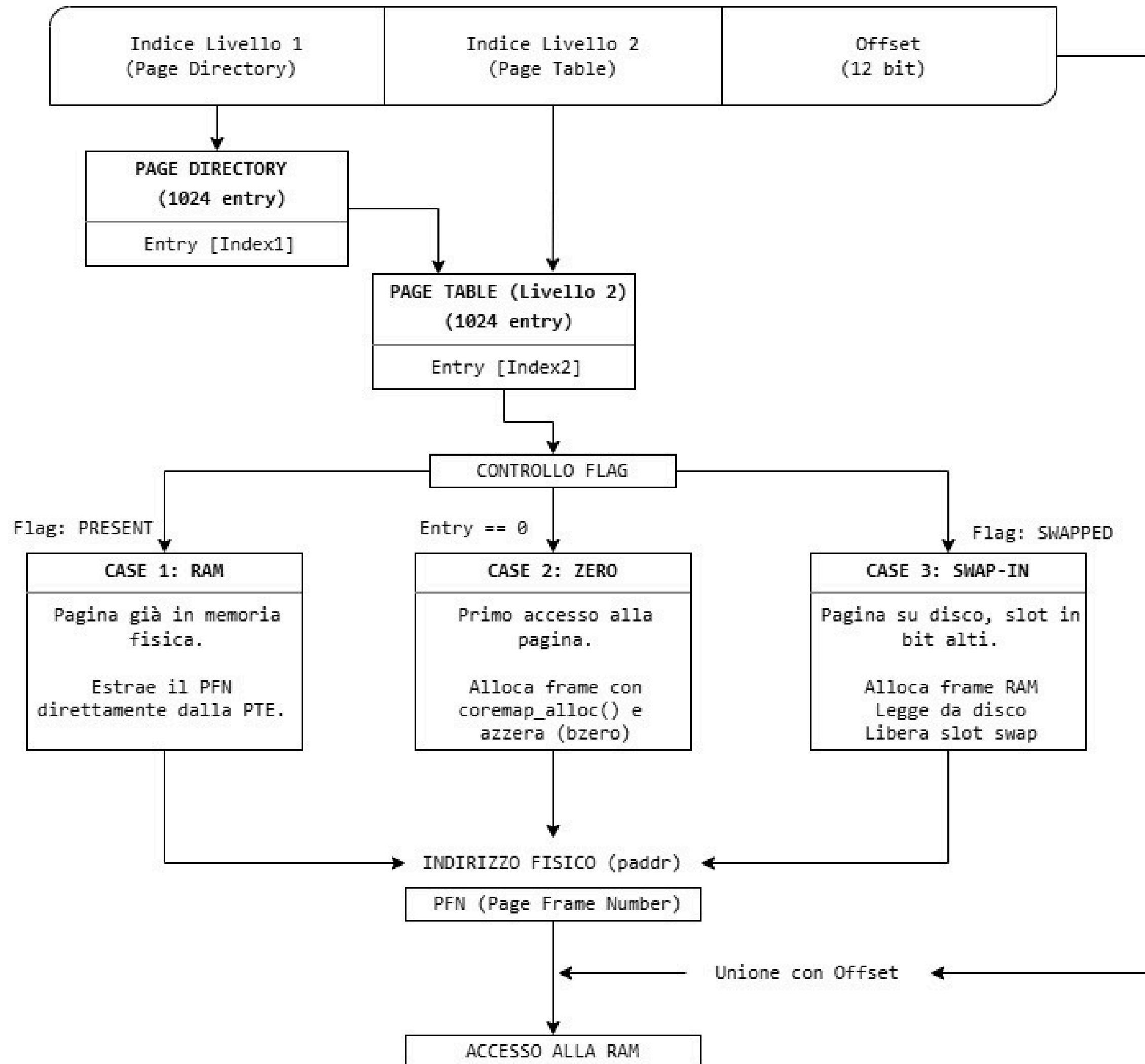
- Valida l'indirizzo e i permessi (readable, writeable, executable) confrontandoli con la lista concatenata delle regioni dell'address space.
- Aggiorna il TLB (gestendo slot vuoti o rimpiazzamento tlb_random).

pagetable_translate:

- Caso 1 (In RAM): Ritorna l'indirizzo fisico.
- Caso 2 (entry = 0): alloca un frame (azzerato con bzero) e chiama coremap_set_owner.
- Caso 3 (In Swap): alloca un frame, esegue swap_read dal disco, libera lo slot sulla bitmap dello swap e aggiorna la PTE a PTE_PRESENT.



Traduzione pagetable



Statistiche benchmark

Test	RAM size	Tempo (s)	TLB fault (free)	TLB fault (replace)	TLB Invalidation	Page fault	Swap read (page in)	Swap write (page out)
ctest	512K	12248.748	92867	32645	92804	124986	124726	124912
ctest	1M	11602.368	9658	116309	9595	123935	123675	123735
ctest	2M	13.022	64	120321	1	260	0	0
forktest	512K	3.795	221	0	220	28	34	34
forktest	1M	0.844	212	0	72	5	0	0
forktest	2M	0.848	214	0	74	5	0	0
huge	512K	292.087	2896	706	2833	3582	3067	3511
huge	1M	270.589	359	3238	296	3423	2908	3225
huge	2M	231.972	170	3398	107	3155	2640	2702
matmult	512K	58.763	631	193	568	813	430	740
matmult	1M	46.859	106	695	43	772	389	572
matmult	2M	0.686	64	732	1	383	0	0

Statistiche benchmark

Test	RAM size	Tempo (s)	TLB fault (free)	TLB fault (replace)	TLB Invalidation	Page fault	Swap read (page in)	Swap write (page out)
malloctest	512K	0.625	7	0	1	7	0	0
malloctest	1M	0.655	7	0	1	7	0	0
malloctest	2M	0.627	7	0	1	7	0	0
palin	512K	15.321	5	0	1	5	0	0
palin	1M	15.321	5	0	1	5	0	0
palin	2M	15.295	5	0	1	5	0	0
parallelVM*	512K	523.375	16858	0	32098	5242	5070	5211
parallelVM	1M	349.415	18806	0	21821	3494	3156	3325
parallelVM	2M	12.830	8615	0	3095	346	5	16
sort	512K	160.582	1354	561	1291	1756	1463	1683
sort	1M	57.310	206	1728	143	814	521	614
sort	2M	3.538	64	1845	1	293	0	0

Analisi dei risultati

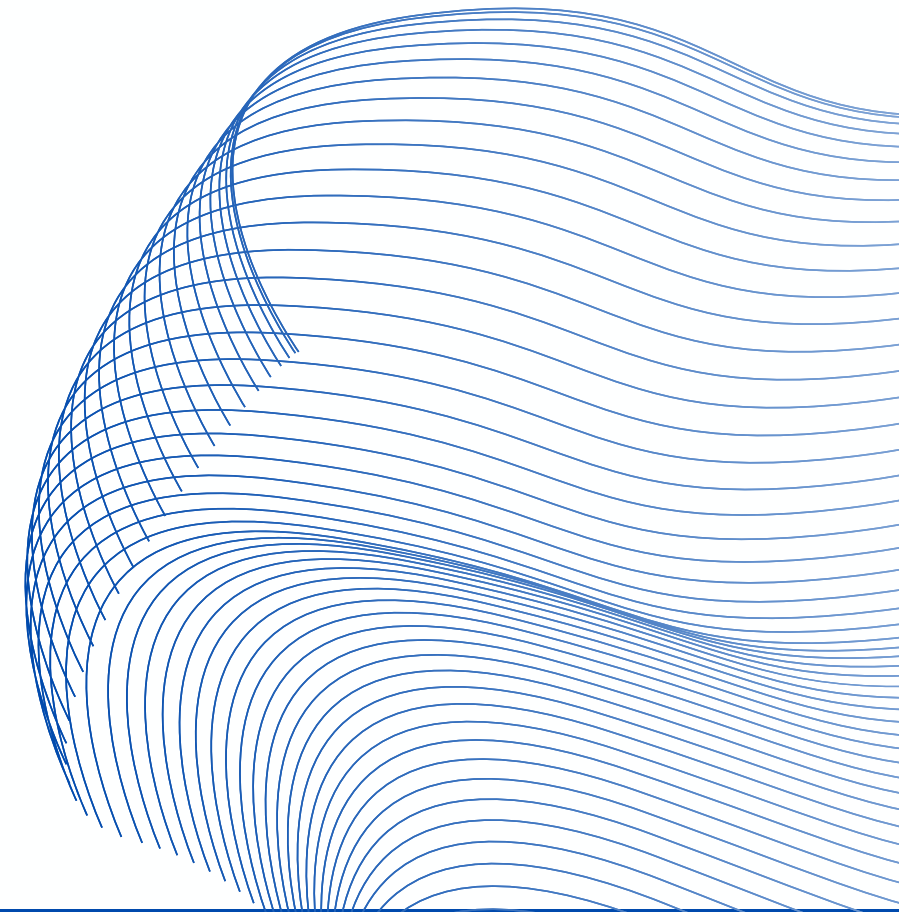
Per fornire un'interfaccia a questo meccanismo, abbiamo introdotto a menu.c il comando **vmstats**, il quale chiama le funzioni definite nel file vmstats.c. Il comando viene invocato scrivendo vmstats <flag>:

- --start: inizia la registrazione delle statistiche sulla VM;
- --stop: termina la registrazione delle statistiche;
- --reset: resetta tutti i contatori delle statistiche;
- --print: stampa le statistiche raccolte;



Il fenomeno del Thrashing (ctest & huge):

- L'accesso a balzi distrugge il principio di località spaziale.
- Sotto i 2MB di RAM, il rimpiazzamento FIFO porta a un continuo I/O su disco (tempo di esecuzione di ctest > 3 ore).



Analisi dei risultati

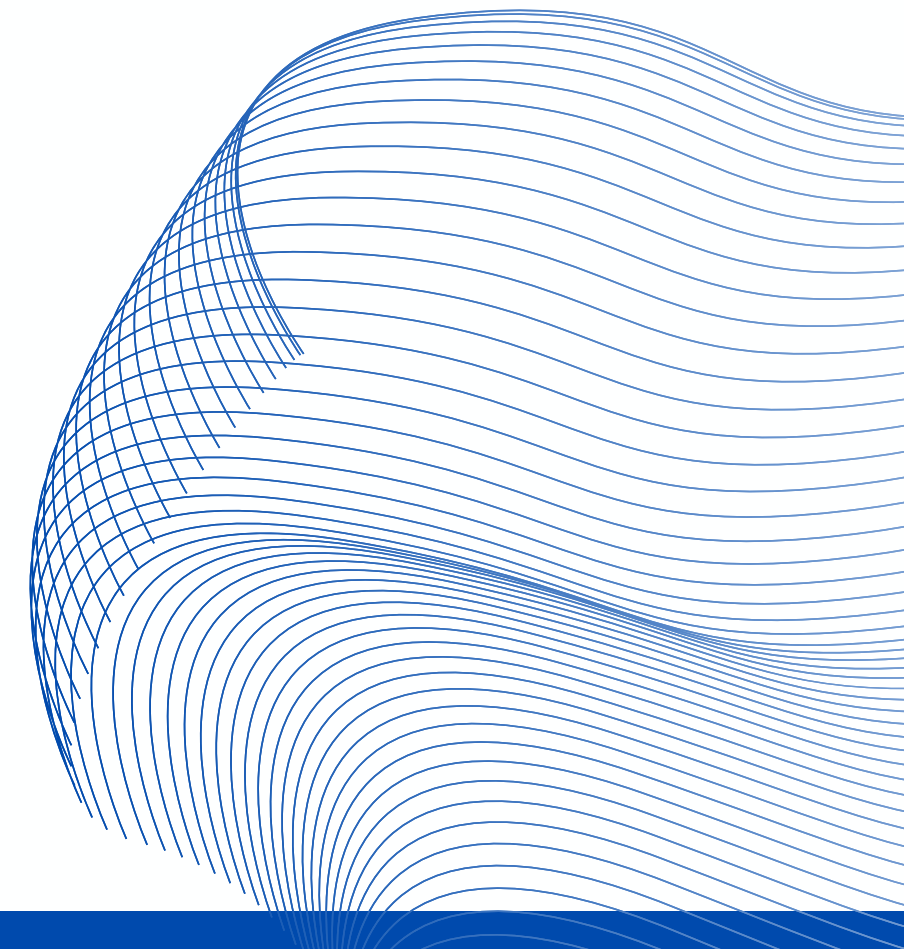
Ottimizzazione e Località Spaziale (sort):

- All'abbassarsi della dimensione delle sotto-partizioni del Quicksort sotto la soglia della RAM fisica, le operazioni di swap si azzerano, validando l'architettura.

I Limiti della Memoria Kernel (parallelVM):

- Con 512K di RAM, 11 processi su 24 falliscono per Out Of Memory. Le pagine adibite allo stack del kernel sono FIXED e non swappabili.

Conclusioni: Il sistema dimostra stabilità in scenari concorrenti e una corretta implementazione della logica on-demand, evidenziando i classici trade-off hardware/software delle politiche FIFO.





**Politecnico
di Torino**
International
University

Grazie per l'attenzione